

Procesamiento de Imágenes

Escuela de Informática y Telecomunicaciones

Universidad Diego Portales

TAREA 1

Profesor: Jorge E. Zambrano.

Fecha de entrega: 11-septiembre-2026

1. Objetivo

Explorar distintas técnicas de compensación de iluminación y mejora de contraste, desde las transformaciones clásicas basadas en tablas de búsqueda (LUT) hasta un modelo convolucional llamado Zero-DCE, y evaluar su efecto sobre una tarea concreta: la detección automática de personas en imágenes nocturnas usando YoloV8.

El propósito de fondo no es encontrar “el mejor método”, sino reconocer la importancia del preprocesamiento de una imagen, y a reconocer cuándo una diferencia observada es real y cuándo es ruido de la medición.

2. Descripción

Se trabajará con nueve imágenes del conjunto de prueba de la base de datos DARK FACE [1], disponibles en la carpeta `people/`. Son escenas nocturnas reales en las que prácticamente toda la información de la imagen se concentra en unos pocos niveles de intensidad bajos.

Estas imágenes no tienen una versión de referencia correctamente expuesta. En consecuencia, no es posible medir la calidad del realce comparando contra una imagen “verdadera”, la evaluación debe hacerse de forma indirecta, aplicando un detector de personas sobre la imagen procesada y observando cómo cambia su desempeño.

Se entrega un cuaderno Jupyter con las funciones necesarias para YoloV8 y Zero-DCE ya implementadas.

3. Métodos de mejora de contraste

Escriba las funciones en un archivo externo llamado `funciones.py` y guárdelo dentro de una carpeta llamada `utils`. Importe las funciones desde el notebook.

Todas las funciones reciben una imagen en formato BGR (tal como la entrega `cv2.imread`) y devuelven la tupla (`imagen_procesada`, `lut`). Además, aceptan una bandera `bgr` que decide dos cosas simultáneamente: dónde se *miden* los niveles de entrada y dónde se *aplica* la tabla resultante.

- `bgr=True`: los niveles se miden sobre los tres canales y la tabla se aplica a cada canal B, G y R por separado.
- `bgr=False`: los niveles se miden sobre el canal de luminancia Y del espacio YCbCr y la tabla se aplica solo ahí, dejando la información de color intacta.

Note que Zero-DCE no admite esta distinción, porque procesa la imagen completa. Tampoco retorna la `lut`, solo la imagen procesada.

Puede usar el código entregado en las clases como base.

Permitido: `cv2.imread`, `cv2.cvtColor`, `cv2.split`, `cv2.merge`, `cv2.calcHist`, `cv2.LUT` y `cv2.createCLAHE`.

Prohibido: `cv2.equalizeHist`, `cv2.normalize` y cualquier función que resuelva el método completo.

3.1 Transformación de potencia (gamma)

Aplica $s = (r/255)^\gamma \cdot 255$ con $\gamma = 0.4$. Al ser $\gamma < 1$, expande los niveles bajos a costa de comprimir los altos. Función: `gamma_transformation(img, gamma, bgr)`.

3.2 Estiramiento lineal de contraste

Mapea linealmente un intervalo $[lo, hi]$ de la entrada al rango completo $[0, 255]$, recortando lo que queda fuera. Se usan dos criterios distintos para elegir ese intervalo como se vio en clase:

- Mínimo y máximo observados en la imagen: `estiramiento_min_max(img, bgr)`.
- Percentiles 5 y 95: `estiramiento_lo_hi(img, 5, 95, bgr)`.

3.3 Ecualización de histograma

Debe aplicar la ecualización del histograma en una función llamada `ecualizacion_histograma(img, bgr)`. Implemente usted mismo la función de ecualización de histograma, puede usar el código entregado en clase. No se permite el uso de la función directa de `cv2`.

3.4 CLAHE

Ecualización adaptativa con limitación de contraste: en lugar de una única tabla global, calcula una tabla por bloque de la imagen. Puede usar la función que proporciona `cv2`. Parámetros `clipLimit=2.0` y `tileGridSize=(8,8)`. Función: `ecualizacion_clahe(img, bgr)`.

3.5 Realce con una red convolucional: Zero-DCE

Zero-DCE [2] es una red de unos 79 mil parámetros que no predice la imagen realzada, sino un conjunto de curvas de ajuste que se aplican de forma iterativa a cada píxel. Puede entenderse como una generalización de los métodos anteriores, en vez de una tabla de búsqueda única para toda la imagen, aprende una curva distinta por píxel.

Esta tarea solo solicita instalar y ejecutar la red. La red se ejecuta con los pesos preentrenados que provee el repositorio oficial; no debe entrenarse nada. Función: `zero_dce(img)`, proporcionada en el cuaderno.

4. Detección de personas

Se utilizará YOLOv8 a través de la librería `ultralytics`, detectando únicamente la clase *person* (clase 0 de COCO) con confianza mínima 0.5. La función `detectar_personas(img, conf)` devuelve el número de detecciones, la confianza media y la imagen con los *bounding boxes* dibujados y la confianza impresa sobre cada uno.

```
from ultralytics import YOLO
modelo_yolo = YOLO("yolov8n.pt")
res = modelo_yolo(img, classes=[0], conf=0.5)[0]
```

Nota: Ultralytics asume orden BGR cuando recibe un arreglo de NumPy, de modo que la imagen se entrega directamente, sin invertir los canales.

5. Experimentos requeridos

5.1. Inspección de una imagen.

Elija una imagen y despléguela junto con su histograma. Analice el histograma y el rango de niveles en el que está concentrada la información.

5.2 Efecto de cada método sobre la imagen de ejemplo.

Usando la imagen escogida previamente, compare cada método con sus dos variantes (`bgr` en `True` y `False`). Analice además la imagen resultante de Zero-DCE. La comparación tiene dos ejes:

1. Comparación entre canales, ¿hay diferencia entre trabajar sobre BGR e Y?
2. Comparación entre métodos, ¿Cuál método funciona mejor para el ejemplo dado?

5.3 Visualización de la transformación.

Por cada método, visualice la imagen resultante, la transformación y los histogramas original y resultante usando la imagen escogida. Incluya además Zero-DCE, pero recuerde que no retorna transformación.

5.4 Tabla de detecciones

Complete la Tabla 1 con el número de personas detectadas en cada imagen, para cada método y cada variante. Además, usando la imagen escogida anteriormente, muestre el resultado de las detecciones para cada método. Para el cuaderno, muestre la tabla ordenada en formato pandas.

Muestre la tabla en orden descendente de acuerdo con el total de personas detectadas.

Método	0.png	1.png	2.png	10.png	17.png	18.png	31.png	94.png	96.png	TOTAL
Imagen original										
Gamma $\gamma = 0.4$ (BGR)										
Gamma $\gamma = 0.4$ (Y)										
Estiramiento min-max (BGR)										
Estiramiento min-max (Y)										
Estiramiento 5-95 (BGR)										
Estiramiento 5-95 (Y)										
Ecualización (BGR)										
Ecualización (Y)										
CLAHE (BGR)										
CLAHE (Y)										
Zero-DCE										

Tabla 1: Número de personas detectadas por método (confianza mínima 0.5). (BGR) indica que la transformación se aplicó a cada canal por separado; (Y), solo sobre la luminancia.

5.5 Tabla de confianza media

Construya una segunda tabla con la misma estructura, pero registrando la confianza media de las detecciones en lugar del conteo (0 si no hubo ninguna detección). La última columna corresponde al promedio de las confianzas. Muestre los resultados en orden descendente. Para el cuaderno, muestre la tabla ordenada en formato pandas.

5.6 Sensibilidad al umbral

Repita la tabla 1 con una confianza mínima de 0.25 y compare el orden de los métodos con el obtenido a 0.5. Analice si hay consistencia entre los métodos variando el umbral. Para el cuaderno, muestre la tabla ordenada en formato pandas.

6. Entregables

- Informe individual en formato PDF, con las Tablas completas. Recuerde que cada resultado debe ir con un análisis de resultados. Si el análisis es básico o no hay análisis, no se entregará puntaje.
- El cuaderno. ipynb ejecutado, con todas sus salidas visibles. El análisis de resultados se debe reportar en el informe, no en el cuaderno. Incluya la carpeta utils con las funciones requeridas.
- La carpeta resultados/parte_a/ con las imágenes anotadas de todos los métodos. En el informe muestre solo los ejemplos solicitados, pero entregue el conjunto completo. Organice los resultados en carpetas.

7. Referencias

- [1] W. Yang, Y. Yuan, W. Ren, J. Liu, W. J. Scheirer, Z. Wang, et al., “Advancing image understanding in poor visibility environments: A collective benchmark study”, IEEE Transactions on Image Processing, vol. 29, pp. 5737–5752, 2020.
- [2] C. Guo, C. Li, J. Guo, C. C. Loy, J. Hou, S. Kwong, R. Cong, “Zero-Reference Deep Curve Estimation for Low-Light Image Enhancement”, IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 1780–1789, 2020.
- [3] R. C. Gonzalez y R. E. Woods, Digital Image Processing, 4.^a ed., Pearson, 2018. Capítulo 3.